Maximum Profit with K Transactions

Name: Ramya S S

Register No: 192421226

Given the daily prices of a stock and a maximum number of allowed transactions, determine the maximum profit that can be achieved. Each transaction consists of buying and selling once. The objective is to maximize profit within the given constraints. Use Dynamic Programming to solve efficiently.

Input Format:

prices[] = stock prices

k = number of transactions

Sample Input:

prices = 10 22 5 75 65 80

k = 2

Expected Output:

Max Profit = 87

AIM

To find the maximum profit from buying and selling a stock with at most K transactions using the Dynamic Programming technique.

INTRODUCTION

The Maximum Profit with K Transactions problem involves finding the highest possible profit from a sequence of daily stock prices. Each transaction consists of **one buy followed by one sell**, and at most **K transactions** are allowed.

Dynamic Programming is used to divide the problem into smaller subproblems and store previously calculated results. This avoids repeated calculations and efficiently determines the maximum profit.

For the given input:

- **Prices:** 10, 22, 5, 75, 65, 80
- **K:** 2
- **Maximum Profit: 87**

ALGORITHM

1. Read the stock prices and the maximum number of transactions K.
2. Let n be the number of days.
3. Create a DP table dp[K+1][n].
4. Initialize the first row and first column to 0, since no profit is possible with zero transactions or one day.
5. For each transaction t from 1 to K:
    - Set maxDiff = -prices[0].
    - For each day d from 1 to n-1:
        - Calculate the profit by selling on day d.
        - Compare it with the profit from doing nothing.
        - Store the maximum value in dp[t][d].
        - Update maxDiff.
6. The value dp[K][n-1] gives the maximum profit.
7. Display the maximum profit.

PSEUDOCODE:

```
MaximumProfit(prices, K)
n = length(prices)
Create dp[K+1][n] and initialize with 0
for t = 1 to K
   maxDiff = -prices[0]
   for d = 1 to n-1
      dp[t][d] = max(dp[t][d-1],
                  prices[d] + maxDiff)
      maxDiff = max(maxDiff,
                  dp[t-1][d] - prices[d])
return dp[K][n-1]
```

CODE:

```
def max_profit(prices, k):
    n = len(prices)

    if n < 2 or k == 0:
        return 0

    dp = [[0] * n for _ in range(k + 1)]

    for t in range(1, k + 1):
        max_diff = -prices[0]

        for d in range(1, n):
            dp[t][d] = max(dp[t][d - 1],
                           prices[d] + max_diff)
            max_diff = max(max_diff,
                           dp[t - 1][d] - prices[d])

    return dp[k][n - 1]



prices = [10, 22, 5, 75, 65, 80]
k = 2

print("Maximum Profit =", max_profit(prices, k))
```

INPUT:

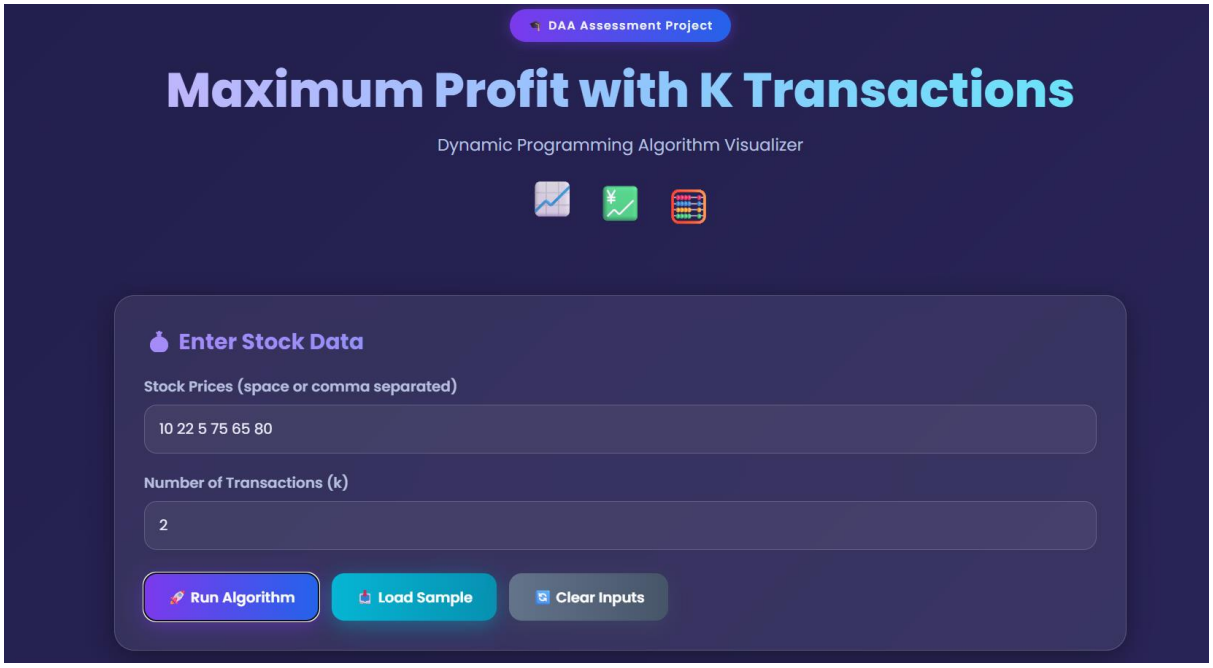
prices = 10 22 5 75 65 80

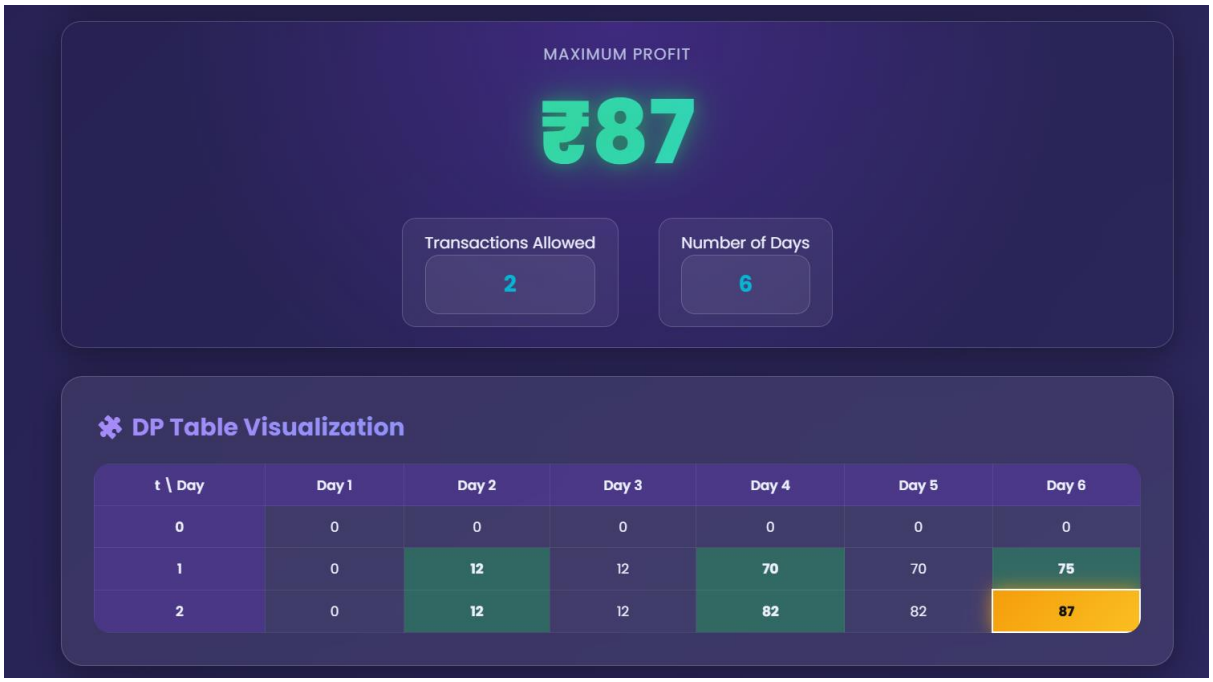
k = 2

OUTPUT:

Max Profit = 87

IMPLEMENTATION





Step-by-Step Algorithm Execution

Step 1: Initialize DP table of size (2+1) x 6 with all zeros.

Step 2: Process Transaction 1: scan days left to right, tracking the best (dp[t-1][d] - price[d]) seen so far as maxDiff.

Step 3: Update dp[1][d] = max(dp[1][d-1], price[d] + maxDiff) for every day d, recording the best profit possible.

Step 4: Process Transaction 2: scan days left to right, tracking the best (dp[t-1][d] - price[d]) seen so far as maxDiff.

Step 5: Update dp[2][d] = max(dp[2][d-1], price[d] + maxDiff) for every day d, recording the best profit possible.

Step 6: Final Answer: dp[2][5] = 87 → Maximum Profit.

Stock Price Timeline

Day 1	→	Day 2	→	Day 3	→	Day 4	→	Day 5	→	Day 6
₹10		₹22		₹5		₹75		₹65		₹80

Transaction Analysis

Transaction 1

Buy → Day 1 (₹10)
Sell → Day 2 (₹22)
Profit = ₹12

Transaction 2

Buy → Day 3 (₹5)
Sell → Day 6 (₹80)
Profit = ₹75

Total Profit = ₹87

How Dynamic Programming Solves This Problem

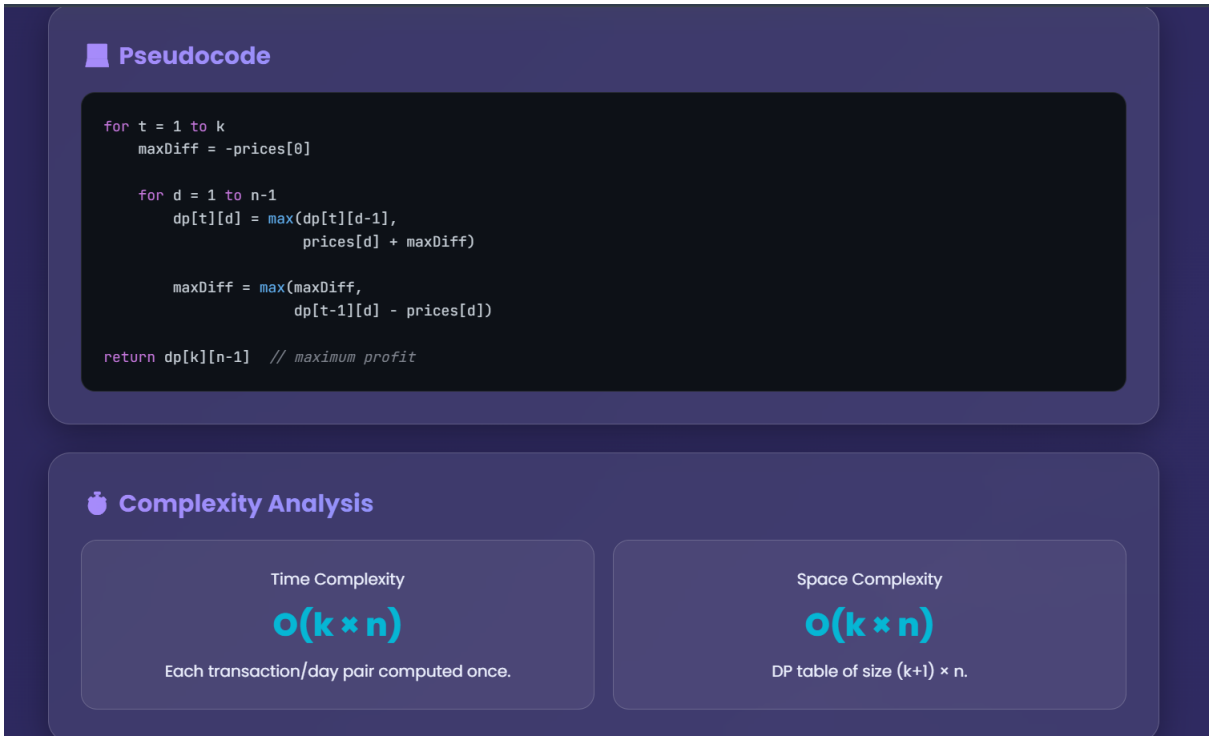
Divide the problem into smaller subproblems: profit up to day *d* using *t* transactions.

DP stores the best profit achievable up to each day for each transaction count.

Reuse previously computed results instead of recomputing them.

Avoid repeated / redundant calculations found in brute-force recursion.

The final DP cell dp[k][n-1] holds the maximum overall profit.



CONCLUSION

The Maximum Profit with K Transactions problem is efficiently solved using Dynamic Programming. By storing the best profit for each number of transactions and each day, repeated calculations are avoided. For the given input, the algorithm produces a maximum profit of 87.